

Script started on 2026-02-07 01:02:18-06:00 [TERM="xterm" TTY="/dev/pts/27" COLUMNS=80]
rc47379@ares:~\$ pwd

/home/students/rc47379
rc47379@ares:~\$ cat imaginary.info

NAME: Calvin Rice

CLASS: CSC122-002

TITLE: What a complex problem

LEVEL: 3

DESCRIPTION:

A class that does various math operations on complex numbers with real and imaginary parts.

imaginary.cpp:

```
1  #include <iostream>
2  #include <cmath>
3
4  class ComplexNumber
5  {
6  private:
7      double real, imaginary;
8  public:
9      ComplexNumber(double _real, double _imaginary)
10     {
11         real = _real;
12         imaginary = _imaginary;
13     }
14
15     ComplexNumber add(ComplexNumber _c)
16     {
17         return ComplexNumber(real + _c.real, imaginary + _c.imaginary);
18     }
19
20     ComplexNumber subtract(ComplexNumber _c)
21     {
22         return ComplexNumber(real - _c.real, imaginary - _c.imaginary);
23     }
24
25     ComplexNumber multiply(ComplexNumber _c)
26     {
27         return ComplexNumber((real * _c.real) - (imaginary * _c.imaginary), (real *
28         _c.imaginary) + (imaginary * _c.real));
29     }
30
31     ComplexNumber divide(ComplexNumber _c)
32     {
33         double denominator = (_c.real * _c.real) + (_c.imaginary * _c.imaginary);
34         if(denominator == 0.0)
35         {
36             return ComplexNumber(0, 0);
37         }
38
39         double newReal = (real * _c.real + imaginary * _c.imaginary) / denominator;
40         double newImaginary = (imaginary * _c.real - real * _c.imaginary) / denominator;
41         return ComplexNumber(newReal, newImaginary);
42     }
43
44     double magnitude()
45     {
46         return sqrt(real * real + imaginary * imaginary);
47     }
48
49     ComplexNumber negate()
50     {
51         return ComplexNumber(-real, -imaginary);
52     }
53
54     ComplexNumber conjugate()
55     {
56         return ComplexNumber(real, -imaginary);
57     }
58
59     double get_real()
60     {
61         return real;
62     }
63
64     double get_imaginary()
65     {
66         return imaginary;
67     }
68
69     void output()
70     {
71         std::cout << real << " + " << imaginary << "i";
72     }
73 };
74
75 int main() {
76     ComplexNumber a(4, 3);
77     ComplexNumber b(1, -2);
78
79     std::cout << "complex number a = ";
80     a.output();
81
82     std::cout << "\ncomplex number b = ";
83     b.output();
84
85     std::cout << "\na + b = ";
86     a.add(b).output();
87
88     std::cout << "\na - b = ";
89     a.subtract(b).output();
90
91     std::cout << "\na * b = ";
92     a.multiply(b).output();
93
94     std::cout << "\na / b = ";
95     a.divide(b).output();
96
97     std::cout << "\nmagnitude of a = ";
98     a.magnitude().output();
99
100    std::cout << "\nmagnitude of b = ";
101    b.magnitude().output();
102
103    std::cout << "\nnegative of a = ";
104    a.negate().output();
105
106    std::cout << "\nconjugate of a = ";
107    a.conjugate().output();
108
109    return 0;
110 }
```

```
32     double denominator = (_c.real * _c.real) + (_c.imaginary * _c.imaginary);
33     if(denominator == 0.0)
34     {
35         return ComplexNumber(0, 0);
36     }
37
38     double newReal = (real * _c.real + imaginary * _c.imaginary) / denominator;
39     double newImaginary = (imaginary * _c.real - real * _c.imaginary) / denominator;
40     return ComplexNumber(newReal, newImaginary);
41 }
42
43 double magnitude()
44 {
45     return sqrt(real * real + imaginary * imaginary);
46 }
47
48 ComplexNumber negate()
49 {
50     return ComplexNumber(-real, -imaginary);
51 }
52
53 ComplexNumber conjugate()
54 {
55     return ComplexNumber(real, -imaginary);
56 }
57
58 double get_real()
59 {
60     return real;
61 }
62
63 double get_imaginary()
64 {
65     return imaginary;
66 }
67
68 void output()
69 {
70     std::cout << real << " + " << imaginary << "i";
71 }
72 };
73
74 int main() {
75     ComplexNumber a(4, 3);
76     ComplexNumber b(1, -2);
77
78     std::cout << "complex number a = ";
79     a.output();
80
81     std::cout << "\ncomplex number b = ";
82     b.output();
83
84     std::cout << "\na + b = ";
85     a.add(b).output();
86
87     std::cout << "\na - b = ";
88     a.subtract(b).output();
89
90     std::cout << "\na * b = ";
91     a.multiply(b).output();
92
93     std::cout << "\na / b = ";
94     a.divide(b).output();
95
96     std::cout << "\nmagnitude of a = ";
97     a.magnitude().output();
98
99     std::cout << "\nmagnitude of b = ";
100    b.magnitude().output();
101
102    std::cout << "\nnegative of a = ";
103    a.negate().output();
104
105    std::cout << "\nconjugate of a = ";
106    a.conjugate().output();
107
108    return 0;
109 }
```

```

86
87     std::cout << "\na - b = ";
88     a.subtract(b).output();
89
90     std::cout << "\na * b = ";
91     a.multiply(b).output();
92
93     std::cout << "\na / b = ";
94     a.divide(b).output();
95
96     std::cout << "\nmagnitude(a) = " << a.magnitude() << std::endl;
97
98     std::cout << "negate(a) = ";
99     a.negate().output();
100
101     std::cout << "\nconjugate(a) = ";
102     a.conjugate().output();
103
104     std::cout << "\nreal part of a: " << a.get_real() << std::endl;
105     std::cout << "imaginary part of a: " << a.get_imaginary() << std::endl;
106
107     return 0;
108 }

```

rc47379@ares:~\$ CPP imaginary

imaginary.cpp***

./imaginary.cpp: In constructor

'ComplexNumber::ComplexNumber(double, double)':

imaginary.cpp:9:3: warning:

'ComplexNumber::real' should be initialized in the member initialization list [-Werror]

```

9 |     ComplexNumber(double _real, double
    |     ~~~~~
    |     ^~~~~~

```

imaginary.cpp:9:3: warning:

'ComplexNumber::imaginary'

should be initialized in the member initialization list

[-Werror]

imaginary.cpp: In member

function 'ComplexNumber

ComplexNumber::divide(ComplexNumber)':

imaginary.cpp:33:20: warning: comparing

floating-point with '==' or '!='

is unsafe [-Wfloat-equal]

```

33 |     if(denominator == 0.0)
    |     ~~~~~^~~~~

```

rc47379@ares:~\$./imaginary.out

```

complex number a = 4 + 3i
complex number b = 1 + -2i
a + b = 5 + 1i

```

```

a - b = 3 + 5i
a * b = 10 + -5i
a / b = -0.4 + 2.2i
magnitude(a) = 5
negate(a) = -4 + -3i
conjugate(a) = 4 + -3i
real part of a: 4
imaginary part of a: 3
rc47379@ares:~$ cat imaginary.tpq

```

Why do your class methods take fewer arguments than you would expect?

-They only take one complex number that is being operated on with the calling object

Does the compiler change when you have 'x+ y' in your program? So should you add:

-The program makes a new object to output so neither inputs are changed

Does this phenomenon extend to the other operations?

-Yes

What kind of value should be returned from the standard math operations (i.e. what

-Complex numbers should be returned from any method that keeps i, and the magnitude

Does your input method prompt the user? Why should it not?

-My input method doesn't exist because there is no reason for the user to input something

Does your output method print anything besides the Complex number (even an endl)? Why

-It shouldn't because the user might want a different formatted output
rc47379@ares
exit

Script done on 2026-02-07 01:02:53-06:00 [COMMAND_EXIT_CODE="0"]